

Peer-to-Peer Networking I — Connections & Protocol

How a node finds peers, opens connections, performs the handshake, and drives each connection through a state machine — the networking core beneath sync.

SUBJECT hathor-core · Part II · the node, end to end

CHAPTER 34 · Part II · The Node

BRANCH study/hathor-core-studies

EDITION 2026 · v1

P2P · ConnectionsManager · HathorProtocol · Peer state machine · Handshake · Twisted Protocol/Factory · PeerEndpoint · Peer discovery · Netfilter · Message dispatch

Table of Contents

Chapter 34 — Peer-to-Peer Networking I: Connections & Protocol	3
34.1 Localization	4
34.2 What it does, and why it exists	5
34.3 The concepts it rests on	6
A state machine, from scratch	6
34.4 The code, walked	7
34.4.1 ConnectionsManager — owner of the peer population	7
34.4.2 HathorProtocol — one object per connection	9
34.4.3 The state machine: HELLO → PEER-ID → READY	11
34.4.4 A worked handshake, end to end	14
34.4.5 The factories — one protocol per connection	14
34.4.6 Peer identity: the four-layer peer	15
34.4.7 Endpoints: where, and optionally who	16
34.4.8 Peer storage: two address books	16
34.4.9 Peer discovery (brief)	16
34.4.10 Netfilter (brief)	17
34.5 How it plugs into the lifecycle	17
Recap	18

Chapter 34 — Peer-to-Peer Networking I: Connections & Protocol

What you'll learn in this chapter

- Why a full node, which is born knowing no one, must discover, connect to, authenticate, and maintain many simultaneous peers — with no central server to coordinate any of it.
- How `ConnectionsManager` owns the whole peer population, tracking each connection through the **connecting** → **handshaking** → **ready** phases, and how it keeps a bounded, rotating set of peers it actively syncs from.
- What a **state machine** is, and how each TCP connection is driven by one `HathorProtocol` object that walks the states `HELLO` → `PEER-ID` → `READY`.
- How an incoming wire message is dispatched to the right handler for the connection's current state — the same dispatch-table idea from Chapter 4, applied to a network protocol.
- Why authentication needs a multi-step handshake: agreeing on the network, proving identity with a key, and only then opening the connection to ledger traffic.
- Where this layer plugs into the node's life: started by `HathorManager.start`, running on the Twisted reactor, and handing each `READY` peer off to a sync agent (Chapter 35).

A node is useless alone. Everything the previous chapters built — the vertex model, storage, verification, consensus — exists to maintain one shared ledger that thousands of independent machines agree on. None of that agreement can happen until the node can talk to other nodes. This chapter is the machinery of "talking": opening connections, proving who is on each end, and keeping a healthy set of peers alive. The content of those conversations once a connection is healthy — downloading the years of history a fresh node is missing — is the next chapter. Here we build the pipe; Chapter 35 fills it.

34.1 Localization

The peer-to-peer layer lives in `hathor/p2p/`. It is one of the largest packages in the codebase (around fifty files), but it splits cleanly into a **core** — connection management, the protocol, the handshake, peer identity — and a **sync** subsystem that runs on top of a healthy connection. This chapter is the core. Sync-v2 (the `sync_v2/` subdirectory and its agent) is Chapter 35.

hathor/	
├─ manager.py	← calls <code>connections.start()</code> (Ch 29)
├─ vertex_handler/	← where received vertices land (Ch 33)
└─ p2p/	◀ YOU ARE HERE (the core)
manager.py	← <code>ConnectionsManager</code> : owns ALL connections
protocol.py	← <code>HathorProtocol</code> (one per connection) + <code>HathorLineReceiver</code>
factory.py	← <code>Server/Client</code> factories: build a protocol per connection
messages.py	← <code>ProtocolMessages</code> enum: the wire vocabulary
peer.py	← <code>PeerInfo</code> / <code>UnverifiedPeer</code> / <code>PublicPeer</code> / <code>PrivatePeer</code>
peer_id.py	← <code>PeerId</code> (a 32-byte hash type)
peer_endpoint.py	← <code>PeerAddress</code> / <code>PeerEndpoint</code> : where + who to dial
peer_storage.py	← <code>Verified</code> / <code>Unverified</code> peer dictionaries
rate_limiter.py	← leaky-bucket throttling
sync_version.py	← <code>SyncVersion</code> enum (V2 is live)
├─ states/	← the per-connection state machine
base.py	← <code>BaseState</code> (<code>cmd_map</code> = message → handler)
hello.py	← <code>HelloState</code> (network/version compatibility)
peer_id.py	← <code>PeerIdState</code> (identity proof)
ready.py	← <code>ReadyState</code> (ping/peers + hands off to sync)
├─ peer_discovery/	← how new peers are found
bootstrap.py	← a static seed list
dns.py	← DNS-seed lookup
├─ netfilter/	← a small connection-firewall (accept/reject hooks)
└─ sync_v2/	← the sync protocol – DEFERRED to Ch 35

Context. `hathor/p2p/` is the node's mouth and ears. Storage and consensus decide what is true; the p2p layer is how a node learns what its peers believe and tells them what it believes. Without it the node is an island with a correct but frozen copy of the past. It is started near the end of `HathorManager.start` (`hathor/manager.py:327`, `self.connections.start()`) and runs for the node's entire life on the Twisted reactor. Everything that arrives from the network — every block, every transaction — enters the node through a connection managed here, and leaves through the vertex-handler pipeline of Chapter 33.

34.2 What it does, and why it exists

A full node is born knowing almost nothing. It has a hard-coded genesis block, perhaps a short list of seed addresses, and otherwise an empty address book. From that standing start it must solve four problems, continuously, for as long as it runs:

1. **Discovery.** Find the addresses of other nodes to try. There is no central directory — by design, since a central directory would be a single point of control and failure for a system whose whole point is decentralization. A node bootstraps from a static seed list and from DNS seeds, then learns more peers from the peers it already has (gossip).
2. **Connection.** Open a TCP connection to a discovered address, or accept one that a remote node opens to us. A node is both a client (it dials out) and a server (it listens). Hundreds of these connections may be open at once.
3. **Authentication.** A TCP connection only proves "something answered at this address." It does not prove the thing is a real Hathor node, that it is on our network (mainnet vs. testnet), that it speaks a protocol version we understand, or who it is. A node must establish all of this before trusting the connection with ledger data. This is the **handshake**.
4. **Maintenance.** Keep the healthy connections alive (detect dead ones, ping idle ones), bound the total (memory and bandwidth are finite), drop misbehaving peers, and — because a node cannot afford to actively sync from every peer at once — keep a small rotating subset of peers as its active sync partners.

Recap — distributed ledger, no central server (full treatment in Ch. 6, §6.3). A distributed ledger is one shared record of history that many computers each hold a full copy of, with no master copy and no coordinator. Every node is an equal peer. The benefit is that there is nothing to seize, bribe, or take offline; the cost is exactly the four problems above — every node must do its own discovery, connection, authentication, and upkeep, because no one is doing it for them. → full treatment in Ch. 6.

The reason this is a whole subsystem, rather than a few socket calls, is that all four problems run concurrently and indefinitely across many connections, each connection at a different stage, all on a single thread. That shape — many independent, long-lived, event-driven conversations — is precisely what Twisted exists for, and what a state machine exists to tame.

34.3 The concepts it rests on

Three ideas underpin this chapter. Two are recaps of earlier deep-dives; the third — the state machine — is introduced here, since this is the first place in the book where the node is one.

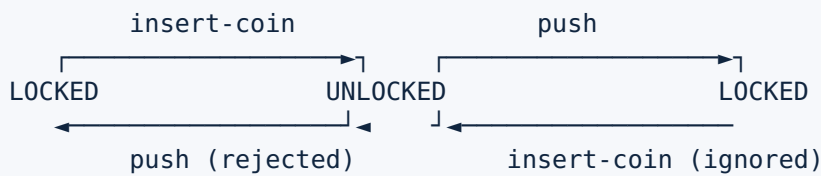
Recap — Twisted: Protocol, Factory, reactor (full treatment in Ch. 16). Twisted is the asynchronous-networking framework the node runs on. Its core, the **reactor**¹, is a single-threaded event loop that waits for network events and calls your code in response. You do not write socket loops; you write a **Protocol**² — an object with callback methods (`connectionMade`, `dataReceived` / `lineReceived`, `connectionLost`) that the reactor invokes as bytes arrive. Because there is one Protocol instance per connection, you need something to manufacture them: a **Factory**³, whose `buildProtocol` method the reactor calls once per new connection. Hathor's protocol is `HathorProtocol`; its factories are `HathorServerFactory` / `HathorClientFactory`. → full treatment in Ch. 16.

Recap — dispatch tables (full treatment in Ch. 4, §4.5). Dispatch means choosing which function to run based on a value, instead of with a long `if/elif` ladder. A **dispatch table** is a dictionary mapping the value to the handler: `handler = table[key]; handler(args)`. Adding a new case is adding a dictionary entry. The p2p layer uses this twice over: a message's type (`HELLO`, `PING`, `PEERS`, ...) is the key, and each state carries its own table of which messages it will answer. → full treatment in Ch. 4.

A state machine, from scratch

A **state machine**⁶ is a way of describing an object whose behavior depends on which phase it is currently in. The object is always in exactly one of a small, fixed set of **states**. Certain events cause **transitions** from one state to another. In each state, the object accepts only a limited set of events; an event that does not belong to the current state is an error.

The textbook example is a coin turnstile. It has two states, `LOCKED` and `UNLOCKED`, and two events, `insert-coin` and `push`:



state	event	action	next state
LOCKED	insert-coin	unlock the arm	UNLOCKED
LOCKED	push	(rejected – stays put)	LOCKED
UNLOCKED	push	let person through	LOCKED
UNLOCKED	insert-coin	(wasted coin)	UNLOCKED

Two things make this design valuable. First, **the legal events are scoped to the state**: in `LOCKED`, pushing does nothing; the turnstile does not have to defend against being pushed because that case is handled by the state, not by a tangle of flags. Second, **the transitions are explicit**: there is exactly one place that says "an inserted coin moves us from `LOCKED` to `UNLOCKED`," so the lifecycle is easy to read and hard to corrupt.

Now scale the idea up. A network connection is also an object with phases. When it first opens, the only sensible message to receive is a greeting; a transaction arriving before the greeting is nonsense (we do not even know who sent it). After the greeting, the only sensible next thing is an identity claim. Only once identity is proven is the connection allowed to carry ledger traffic. That is three states — `HELLO`, `PEER-ID`, `READY` — and the transitions between them are exactly the steps of the handshake. Modelling the connection as a state machine means each state can hold its own dispatch table of acceptable messages, and a message that does not belong to the current phase is automatically rejected. We will see Hathor's version is almost a literal transcription of the turnstile table, with three states instead of two.

34.4 The code, walked

We build the picture from the outside in: first the manager that owns every connection, then the single protocol object that drives one connection, then the state machine and handshake inside it, then the supporting cast (peer identity, endpoints, discovery, netfilter).

34.4.1 `ConnectionsManager` — owner of the peer population

`ConnectionsManager` (`hathor/p2p/manager.py:75`) is the one long-lived object that holds all connections and all knowledge of peers. There is exactly one per node; the `HathorProtocol` for each connection holds a reference back to it (`protocol.py:115`). It is built and wired by the builder (Chapter 24) and handed the reactor, the node's own identity (`my_peer`), the pub-sub bus, and a bundle of flags.

Its most informative fields are the **peer sets**, because they are the connection lifecycle made concrete (`manager.py:148–165`):

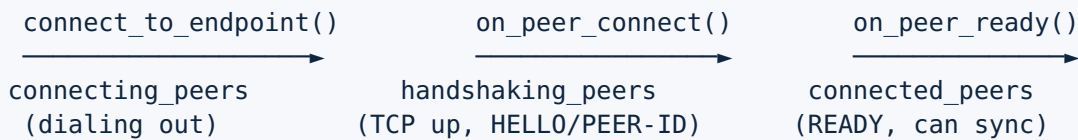
```
# All connections, in any phase.
self.connections = set()          # set[HathorProtocol]

# Outbound dials in progress (TCP not yet established).
self.connecting_peers = {}        # endpoint -> _ConnectingPeer

# TCP up, but still doing the handshake (not yet authenticated).
self.handshaking_peers = set()    # set[HathorProtocol]

# Handshake complete: authenticated and ready to communicate.
self.connected_peers = {}         # PeerId -> HathorProtocol
```

A connection moves left-to-right through these sets, and the move is the lifecycle:



- A peer enters `connecting_peers` when we start dialing it (`connect_to_endpoint` , `manager.py:743`); the value is a `_ConnectingPeer` holding the target endpoint and the Twisted `Deferred`⁴ for the in-progress connection attempt.
- When the TCP connection is established, the protocol's `on_connect` fires and calls back into `on_peer_connect` (`manager.py:413`), which adds the protocol to `connections` and `handshaking_peers` . It also enforces `max_connections` here (`manager.py:415`): if the node is already full, the new connection is dropped immediately.
- When the handshake finishes and the peer reaches READY, `on_peer_ready` (`manager.py:428`) removes it from `handshaking_peers` and records it in `connected_peers` , keyed by the now-verified `PeerId` . Note the key change: a connecting/handshaking peer is tracked by its connection object because we do not yet trust its identity; only a ready peer is keyed by its `PeerId` , because only now has that id been proven.

Each transition also publishes an event on the pub-sub bus — `NETWORK_PEER_CONNECTING` , `NETWORK_PEER_CONNECTED` , `NETWORK_PEER_READY` , `NETWORK_PEER_DISCONNECTED` (e.g. `manager.py:422` , `:439` , `:502`) — so that metrics, the status API, and the event subsystem (Chapter 30) can observe the peer population without the manager knowing who is listening.

The connection loops. `start` (`manager.py:292`) does not connect to everyone at once; that would be a thundering herd. Instead it arms several `LoopingCall`⁵ timers on the reactor:

- `lc_connect` (`manager.py:186`, every 0.2 s) pulls one new peer to dial from a queue (`connect_to_peer_from_connection_queue`, `manager.py:549`). The queue is fed by the peers our existing peers tell us about — this is the gossip mechanism in action.
- `lc_reconnect` (`manager.py:171`, every 5 s) retries known-but-disconnected peers and prunes peers we have not seen in a long time (`peers_cleanup`, `manager.py:534`).
- `lc_sync_update` (`manager.py:175`) periodically rotates which peers we actively sync from (below).
- `start` finally calls `do_discovery` (`manager.py:271`) to kick off the discovery strategies.

Sync rotation. A node may be connected to hundreds of peers, but it cannot afford to download history from all of them simultaneously. So only a bounded subset — `MAX_ENABLED_SYNC` peers — has sync enabled at any moment, and the manager periodically reshuffles that subset so no peer is starved. When a peer first becomes ready, sync is enabled for it only if we are still under the cap (`manager.py:464`). The reshuffle is `_sync_rotate_if_needed` (`manager.py:908`): it shuffles the connected peers, picks a new random `MAX_ENABLED_SYNC` set, and enables/disables sync on connections to match (`manager.py:927–931`). A small `always_enable_sync` set (`manager.py:180`) lets an operator pin specific peers as permanent sync partners.

Recap — sync agents exist, detail is Ch 35. Each READY connection owns a **sync agent**: the object that runs the actual ledger-download conversation. The manager's job is only to decide which connections have an active sync agent at any time (the rotation above) and to relay a freshly-created transaction to all ready peers (`send_tx_to_peers`, `manager.py:378`). What the sync agent does — streaming blocks and transactions in dependency order — is the canonical sync-v2 deep-dive in Chapter 35. Here, treat "enable sync on this connection" as a black box. → full treatment in Ch. 35.

34.4.2 HathorProtocol — one object per connection

Where `ConnectionsManager` is the city, `HathorProtocol` (`hathor/p2p/protocol.py:51`) is one street: a single object created for, and living exactly as long as, one connection. Its docstring states the contract plainly: "An instance of this class is created for each connection." It holds that connection's transport, its current state, its rate limiters, its idle-timeout timer, and — once the handshake succeeds — the verified `PublicPeer` on the other end (`protocol.py:86`, exposed through the `peer` property at :99 which asserts the peer is set).

`HathorProtocol` is split from the Twisted plumbing on purpose. The bytes-on-the-wire mechanics live in a subclass, `HathorLineReceiver` (`protocol.py:430`), which inherits both `HathorProtocol` and Twisted's `LineReceiver`. This is the Hathor wire format in one sentence: **one text message per line**. `LineReceiver` buffers incoming bytes and calls `lineReceived` once per complete line, so the protocol never has to deal with partial reads.

The three Twisted lifecycle callbacks (Chapter 16) map onto Hathor's own methods:

```
class HathorLineReceiver(LineReceiver, HathorProtocol):
    MAX_LENGTH = 65536

    def connectionMade(self) -> None:          # protocol.py:436
        super().connectionMade()
        self.setLineMode()
        self.on_connect()                       # -> enter HELLO state

    def connectionLost(self, reason=connectionDone) -> None: # :441
        super().connectionLost()
        self.on_disconnect(reason)             # -> tell the manager
```

`on_connect` (`protocol.py:263`) records the time, arms the idle-timeout timer, sets the initial state to `HELLO` (`protocol.py:275`), and notifies the manager via `on_peer_connect`. `on_disconnect` (`protocol.py:296`) tears the state down and notifies the manager via `on_peer_disconnect`. The `MAX_LENGTH = 65536` cap (`protocol.py:434`) is a defense: a peer cannot make us buffer an unbounded line.

Sending and receiving a message. A message is a command word, optionally followed by a space and a payload. `send_message` on the line receiver builds the line and writes it (`protocol.py:479`):

```
def send_message(self, cmd_enum, payload=None):    # protocol.py:479
    cmd = cmd_enum.value
    line = '{} {}'.format(cmd, payload).encode('utf-8') if payload else cmd.encode('utf-8')
    self.sendLine(line)
```

Receiving is the mirror image and the heart of dispatch. `lineReceived` (`protocol.py:450`) decodes the bytes, splits off the first word as the command, looks it up in the `ProtocolMessages` enum, and forwards to `recv_message`:

```
def lineReceived(self, line: bytes) -> None:      # protocol.py:450
    if self.aborting:                             # connection being torn down; ignore buffered
        return
    sline = line.decode('utf-8')                   # bad UTF-8 -> drop connection
    msgtype, _, msgdata = sline.partition(' ')
    cmd = ProtocolMessages(msgtype)                # unknown command -> drop connection
    self.recv_message(cmd, msgdata)
```

`recv_message` (`protocol.py:322`) is where the dispatch table and the state machine meet:

```

def recv_message(self, cmd: ProtocolMessages, payload: str) -> None: # protocol.py:322
    assert self.state is not None
    ...
    if not self.ratelimit.add_hit(self.RateLimitKeys.GLOBAL): # too many messages?
        self.state.send_throttle(...)
        return
    cmd_handler = self.state.cmd_map.get(cmd) # the dispatch
    if cmd_handler is None:
        self.send_error_and_close_connection('Invalid Command: ...') # not legal in this state
        return
    deferred_result = defer.maybeDeferred(cmd_handler, payload) # handlers may be sync or
    deferred_result.addCallback(lambda _: self.reset_idle_timeout()) \
        .addErrback(self._on_cmd_handler_error, cmd)

```

Read that carefully, because three of the chapter's ideas converge in those lines. First, **the handler comes from `self.state.cmd_map`** — the current state's dispatch table — so the same command word can be legal in one state and rejected in another. Second, a command with no handler in the current state is a protocol violation and the connection is closed; this is the turnstile's "push while locked" made real. Third, `defer.maybeDeferred` lets a handler be a plain function or a coroutine returning a `Deferred` — necessary because some handlers (identity validation) must do asynchronous work like a DNS lookup, while others (ping) are instant.

34.4.3 The state machine: HELLO → PEER-ID → READY

The states are named in an inner enum on the protocol (`protocol.py:70`), and the values are the state classes:

```

class PeerState(Enum): # protocol.py:70
    HELLO = HelloState
    PEER_ID = PeerIdState
    READY = ReadyState

```

`change_state` (`protocol.py:190`) is the single transition function. It lazily instantiates the state class the first time it is entered, calls `on_exit` on the old state and `on_enter` on the new one, and stores it as `self.state`. There is exactly one place a connection's phase changes — the turnstile virtue of "transitions are explicit."

Every state subclasses `BaseState` (`hathor/p2p/states/base.py:30`), which holds the dispatch table and a couple of always-legal handlers:

```

class BaseState: # base.py:30
    cmd_map: dict[ProtocolMessages, Callable[...]]
    def __init__(self, protocol, settings):
        ...
        self.cmd_map = { # base.py:41
            ProtocolMessages.ERROR: self.handle_error,
            ProtocolMessages.THROTTLE: self.handle_throttle,
        }
    def on_enter(self) -> None: raise NotImplementedError
    def on_exit(self) -> None: pass

```

`ERROR` and `THROTTLE` are legal in every state (any peer may report an error or a throttle at any time). Each concrete state then adds the messages it accepts. This is the structure of the whole machine in one diagram:

STATE	on_enter sends	accepts (cmd_map)	on success -> next
HELLO (compat)	HELLO	HELLO (+ ERROR, THROTTLE)	PEER-ID
PEER-ID (identity)	PEER-ID	PEER-ID, READY (+ ERROR, THROTTLE)	READY (when BOTH sides ready)
READY (operate)	PEERS, PING, GET-PEERS ...	PING, PONG, GET-PEERS, PEERS, + sync messages (+ ERROR, THROTTLE)	(terminal: stays READY, sync agent runs here)

Each state below is one class; each transition is one `change_state` call.

HelloState (`hathor/p2p/states/hello.py:35`) checks **compatibility**. On entry it sends a `HELLO` message describing this node (`send_hello`, `hello.py:76`): the app version, the **network name** (`mainnet/testnet/...`), a short hash of the genesis block, a timestamp, the node's capabilities, and the sync protocol versions it supports (`_get_hello_data`, `hello.py:46`). `handle_hello` (`hello.py:83`) validates the peer's matching message and refuses the connection on any mismatch:

- wrong network → `close` (`hello.py:138`); a mainnet node must never talk ledger to a testnet node.
- different genesis short hash → `close` (`hello.py:142`); a different genesis means a different chain entirely.
- clocks too far apart → `close` (`hello.py:146`); timestamps anchor proof-of-work difficulty, so wildly skewed clocks are unusable.
- no sync version in common → `close` (`hello.py:123`); if we cannot agree on how to sync, the connection is pointless.

Only if everything matches does it transition:

`protocol.change_state(protocol.PeerState.PEER_ID)` (`hello.py:173`). Note what is absent: nothing here proves who the peer is. `HELLO` establishes "we are the same kind of node on the same network speaking a common version." Identity is the next state's job — which is exactly why it is a separate state.

PeerIdState (`hathor/p2p/states/peer_id.py:32`) checks **identity**. On entry it sends a `PEER-ID` message containing this node's `id`, its public key, and its endpoints (`send_peer_id`, `peer_id.py:86`). `handle_peer_id` (`peer_id.py:92`) — note it is `async`, because it may do a DNS lookup — validates the peer's claim:

```

async def handle_peer_id(self, payload: str) -> None:          # peer_id.py:92
    peer = PublicPeer.create_from_json(data)                   # parse id + public key
    if protocol.expected_peer_id and peer.id != protocol.expected_peer_id:
        protocol.send_error_and_close_connection('Peer id different from the requested one.')
    ...
    if protocol.connections.is_peer_connected(peer.id):        # already connected to this id?
        protocol.send_error_and_close_connection('We are already connected.')
    entrypoint_valid = await peer.info.validate_entrypoint(protocol)  # the address checks out?
    if protocol.use_ssl:
        certificate_valid = peer.validate_certificate(protocol)      # TLS cert key == claimed
    protocol._peer = peer                                         # accepted; remember the verified peer
    self.send_ready()

```

The identity check is cryptographic, and worth understanding precisely. A peer's `id` is not a self-chosen nickname — it is the double-SHA-256 hash of its RSA public key (`_calculate_peer_id`, `hathor/p2p/peer.py:93`). When `PublicPeer.create_from_json` runs, it calls `validate` (`peer.py:376`), which recomputes the id from the supplied public key and rejects the peer if it does not match (`peer.py:378`). So a peer cannot claim an id without also presenting the public key that hashes to it. And because the TLS connection's certificate is signed with the matching private key, `validate_certificate` (`peer.py:332`) confirms the party on the wire actually holds that private key — not merely a copy of someone's public key. Together these mean: **the id is bound to a keypair, and the connection proves possession of the private half**. That is what makes a `PeerId` a real identity rather than a label.

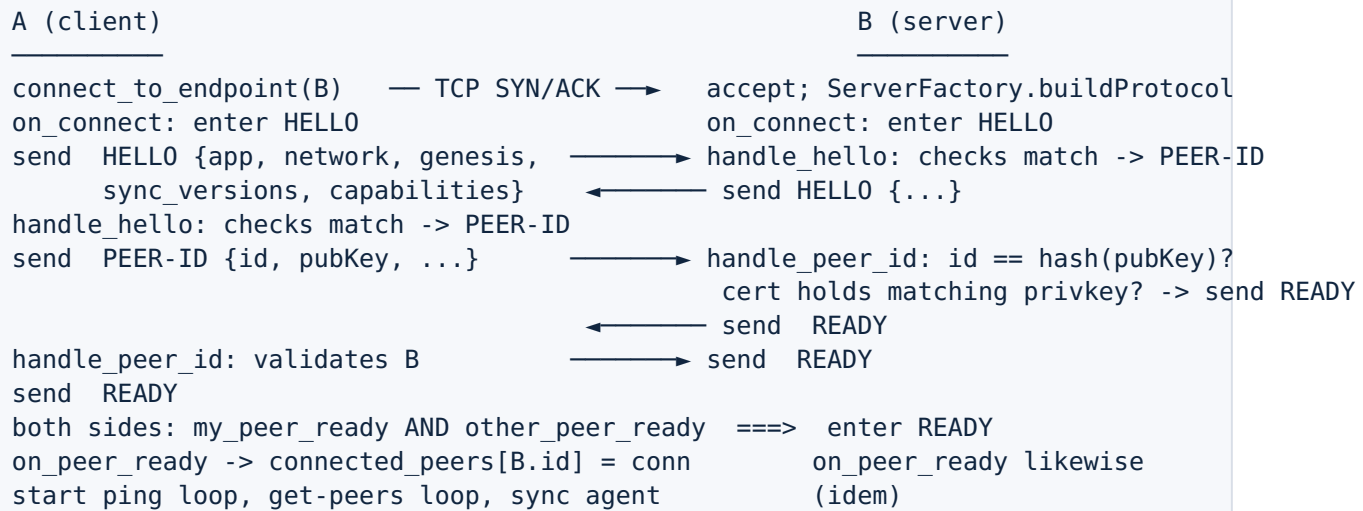
The transition to `READY` is the one genuinely subtle part of the handshake, because it is symmetric. Each side independently finishes validating the other and sends a `READY` message; a side only moves to the `READY` state once it has both sent its own `READY` (`my_peer_ready`) and received the peer's (`other_peer_ready`) (`peer_id.py:54–70`). This double-gate guarantees neither side starts sending ledger traffic before the other is prepared to receive it.

ReadyState (`hathor/p2p/states/ready.py:40`) is **operation**. The handshake is over; the connection is now a trusted, bidirectional channel. On entry (`on_enter`, `ready.py:132`) it notifies the manager (`on_peer_ready`), starts a periodic `PING / PONG` keepalive (`lc_ping`), starts a periodic `GET-PEERS` request to keep learning new peers (`lc_get_peers`, every 5 minutes), and — crucially — **creates and starts the sync agent** (`ready.py:144`, `self.sync_agent.start()`). Its `cmd_map` (`ready.py:83`) now accepts the operational messages: `PING`, `PONG`, `GET-PEERS`, `PEERS`, plus optional capability-gated commands, plus all the sync messages, which it merges in from the sync agent (`ready.py:130`, `self.cmd_map.update(self.sync_agent.get_cmd_dict())`).

This last line is where the two layers join. The sync agent contributes its own message handlers into the ready state's dispatch table. From `recv_message`'s point of view nothing special happens — it is still "look up the command in `self.state.cmd_map`" — but now a `BLOCKS` or `TRANSACTION` message resolves to a sync-agent handler. The core protocol does not know or care what those handlers do; that is Chapter 35.

34.4.4 A worked handshake, end to end

Trace a single outbound connection from nothing to READY. We are node **A**, dialing node **B**.



Every arrow is one line of UTF-8 text. Every box is a `cmd_map` lookup in `recv_message` resolving to a state handler. The whole exchange is four message types — `HELLO`, `PEER-ID`, `READY`, and then the operational set — and the connection only carries a block or a transaction after the final transition. The handshake is multi-step not by accident but because each step gates the next: you cannot meaningfully check identity before you have agreed you are the same kind of node, and you must not carry ledger data before identity is proven.

34.4.5 The factories — one protocol per connection

The reactor needs a way to manufacture a fresh `HathorProtocol` for each new connection, inbound or outbound. That is the `Factory` (`hathor/p2p/factory.py`). Both factories share a base whose `buildProtocol` constructs the line receiver and stamps it with the connection's direction:

```
class _HathorLineReceiverFactory(ABC, protocol.Factory): # factory.py:26
    inbound: bool
    def buildProtocol(self, addr) -> HathorLineReceiver: # factory.py:43
        p = HathorLineReceiver(my_peer=self.my_peer, p2p_manager=self.p2p_manager,
                               use_ssl=self.use_ssl, inbound=self.inbound, settings=self._setting
        p.factory = self
        return p

class HathorServerFactory(_HathorLineReceiverFactory, protocol.ServerFactory): # :55
    inbound = True # we accepted the connection
class HathorClientFactory(_HathorLineReceiverFactory, protocol.ClientFactory): # :61
    inbound = False # we dialed out
```

The only difference between the two is the `inbound` flag, but it matters: the direction is recorded on every protocol and is later used to break ties when two nodes accidentally connect to each other twice (`get_connection_to_drop`, `manager.py:812` — keep the

connection initiated by the peer with the larger id, deterministically, so both sides agree which to drop). The manager builds one of each factory in its constructor (`manager.py:134`); the server factory is handed to `listen` and the client factory to `connect_to_endpoint`.

34.4.6 Peer identity: the four-layer peer

A peer is modelled by four small classes that build on one another (`hathor/p2p/peer.py`). The module docstring is unusually emphatic that they **must not be mixed**, and the design is deliberate (`peer.py:14–31`):

PeerInfo	entrypoints + reconnect bookkeeping	(<code>peer.py:104</code>)
▲ held by		
UnverifiedPeer	a PeerId + PeerInfo	(<code>peer.py:207</code>)
▲ wraps		
PublicPeer	an UnverifiedPeer + a public key	(<code>peer.py:269</code>)
▲ wraps		
PrivatePeer	a PublicPeer + the private key	(<code>peer.py:393</code>)

The split encodes how much we trust this peer's identity in the type itself:

- **UnverifiedPeer** is a peer we have only heard about — typically gossiped to us by another peer via a `PEERS` message. We have an id and some addresses, but no proof either is real. We may try to connect to it; we may not trust it.
- **PublicPeer** is a peer whose public key we hold and whose id we have therefore verified (the id is the hash of the key). This is what `handle_peer_id` produces and what lands in `connected_peers`. We can verify its signatures.
- **PrivatePeer** is us. It additionally holds the private key, so it can sign messages and present a TLS certificate. Each node has exactly one **PrivatePeer** — `my_peer`.

Critically, these are **not** a subclass chain — `PublicPeer` is not a subclass of `UnverifiedPeer` (`peer.py:25`). That is intentional: it means a function typed to take an `UnverifiedPeer` will not silently accept a `PublicPeer`, so the trust level cannot be smuggled across by accident. To go "down" a level you call an explicit converter like `to_unverified_peer()`. The types are a tripwire against treating an unverified rumor as a verified identity.

`PeerId` itself (`hathor/p2p/peer_id.py:18`) is the thinnest of wrappers — a subclass of the project's `Hash` type (`hathor/types.py:30`), i.e. a 32-byte value with convenient `hex` / `repr`. `PeerInfo` (`peer.py:104`) holds the peer's entrypoints (addresses others can reach it at) and the exponential-backoff reconnect state (`retry_interval` doubles on each failure, `increment_retry_attempt`, `peer.py:178`) so a dead peer is retried less and less often instead of being hammered.

34.4.7 Endpoints: where, and optionally who

A `PeerAddress` (`hathor/p2p/peer_endpoint.py:75`) is a frozen `(protocol, host, port)` triple — purely where to connect, e.g. `tcp://1.2.3.4:40403`. A `PeerEndpoint` pairs an address with an optional `peer_id` — where, and who we expect to find there. The distinction matters during dialing: if we connect to an endpoint that names an expected peer id, `handle_peer_id` enforces that the peer who answers actually has that id (`peer_id.py:114`), defeating an attacker who hijacks an address.

`PeerAddress.to_client_endpoint` (used at `manager.py:730`) turns the address into a Twisted client endpoint the reactor can dial. The module even forbids comparing a `PeerAddress` to a `PeerEndpoint` directly (`peer_endpoint.py:31`), again using types to prevent a where/who confusion.

34.4.8 Peer storage: two address books

The manager keeps two peer dictionaries, mirroring the trust split (`hathor/p2p/peer_storage.py`). `VerifiedPeerStorage` (`peer_storage.py:92`) holds `PublicPeer`s we have connected to and authenticated; it lives on the manager (`manager.py:165`).

`UnverifiedPeerStorage` (`peer_storage.py:100`) holds `UnverifiedPeer`s — rumors — and lives on each protocol (`protocol.py:174`), because the peers a connection tells us about are only as trustworthy as that connection. Both are `dict[PeerId, ...]` subclasses with a `max_size`; when full, they evict a random entry (`_ensure_max_size`, `peer_storage.py:44`) so a peer cannot exhaust our memory by flooding us with fake addresses. The flow of a gossiped peer is: arrives in a `PEERS` message → `on_receive_peer` adds it to the protocol's unverified storage (`protocol.py:389`) → the connect loop later picks it as a dial candidate → if the handshake succeeds, it graduates to the manager's verified storage (`on_peer_ready`, `manager.py:431`).

34.4.9 Peer discovery (brief)

Discovery answers "where do I find peers in the first place?" It is a small strategy hierarchy under `peer_discovery/`. `PeerDiscovery` (`peer_discovery/peer_discovery.py:21`) is an abstract base with one method, `discover_and_connect`, that is handed a `connect_to_endpoint` callback and is expected to call it for each peer it finds. Two implementations ship:

- `BootstrapPeerDiscovery` (`peer_discovery/bootstrap.py:27`) is the simplest possible: a static list of entrypoints from configuration; it just calls the callback for each.
- `DNSPeerDiscovery` (`peer_discovery/dns.py:35`) looks up DNS seed hostnames and turns the A/AAAA/TXT records into entrypoints — the same trick Bitcoin uses, letting the seed operator update the peer list without a software release.

The manager holds a list of these strategies (`manager.py:126`) and runs them all in `do_discovery` (`manager.py:271`), once at startup and again periodically when peers run low. After bootstrap, the dominant source of new peers is gossip (`GET-PEERS / PEERS`), so discovery seeds the network and gossip sustains it.

34.4.10 Netfilter (brief)

`netfilter/` is a small, iptables-flavored connection firewall: named chains of rules, each rule a match plus a verdict (accept/reject). Three chains are defined (`netfilter/instances.py:19–22`) and consulted at three points in the connection's life: `pre_conn` before a connection is even built (`NetfilterFactory.buildProtocol` , `netfilter/factory.py:39`), `post_hello` after the HELLO exchange (`hello.py:168`), and `post_peerid` after identity is proven (`peer_id.py:157`). By default every chain's policy is `accept` (`instances.py:20`), so the filter is inert unless an operator adds rules — but the hooks let a node reject, say, a banned address before wasting any work on it.

34.5 How it plugs into the lifecycle

The p2p core is one cog in the node's startup and steady-state machinery described in Chapter 0.

Startup. The builder (Chapter 24) constructs the `ConnectionsManager` , registers the `sync-v2` factory on it, and hands it to the `HathorManager` . Near the end of `HathorManager.start` , after storage is unlocked and metrics are up, the node calls `self.connections.start()` (`hathor/manager.py:327`). That arms the connect/reconnect/sync-update loops and runs discovery — from this instant the node is reaching out to the network.

Steady state. Everything thereafter is reactor-driven (Chapters 16 & 23). The reactor accepts inbound connections (via `listen` , building protocols from the server factory) and completes outbound ones (via `connect_to_endpoint` , the client factory); each new connection gets a fresh `HathorProtocol` that walks `HELLO → PEER-ID → READY` . Reaching `READY` enables a sync agent on a rotating subset of peers. From then on the connection carries `PING / PONG` keepalives, `GET-PEERS / PEERS` gossip, and sync traffic.

Where vertices go. When a sync agent (or a relayed transaction) pulls a new block or transaction off a `READY` connection, it does not apply it directly. It hands the vertex to the vertex handler (Chapter 33), which runs the `verify → consensus → store` pipeline; only then is the vertex part of the node's ledger. Conversely, when this node creates or accepts a new vertex, `send_tx_to_peers` (`manager.py:378`) shuffles the ready connections and pushes it out so the network converges. The p2p layer is thus the boundary: ledger data crosses into the node here and crosses out here, and the vertex handler is the gate in between.

Recap

Component	Responsibility	File (key line)
ConnectionsManager	Owns all connections; runs connect/reconnect/discovery/sync-rotation loops	p2p/manager.py:75
Peer sets	connecting → handshaking → connected — the lifecycle made concrete	p2p/manager.py:148–165
HathorProtocol	One per connection; holds state, transport, peer; dispatches messages	p2p/protocol.py:51
HathorLineReceiver	Twisted plumbing: one message per UTF-8 line	p2p/protocol.py:430
recv_message	Looks up the command in the current state's cmd_map (dispatch)	p2p/protocol.py:322
PeerState enum	The three states HELLO / PEER_ID / READY	p2p/protocol.py:70
HelloState	Compatibility: network, genesis, version, clock	p2p/states/hello.py:35
PeerIdState	Identity: id == hash(pubkey), cert holds privkey; symmetric READY	p2p/states/peer_id.py:32
ReadyState	Operation: ping/peers; creates and starts the sync agent	p2p/states/ready.py:40
HathorServerFactory / HathorClientFactory	Build one protocol per inbound / outbound connection	p2p/factory.py:55 / :61
UnverifiedPeer / PublicPeer / PrivatePeer	Trust levels encoded as distinct (non-inheriting) types	p2p/peer.py:207 / :269 / :393
PeerId	A peer's identity = double-SHA-256 of its public key	p2p/peer.py:93 , peer_id.py:18
PeerEndpoint	Where + optionally who to dial	p2p/peer_endpoint.py

Component	Responsibility	File (key line)
Verified / Unverified storage	Two address books matching the trust split	p2p/peer_storage.py: 92 / :100
ProtocolMessages	The wire vocabulary (HELLO , PEERS , PING , ... + sync)	p2p/messages.py:45
Peer discovery	Bootstrap list + DNS seeds; gossip sustains it	p2p/peer_discovery/
Netfilter	Optional accept/reject hooks at 3 connection stages	p2p/netfilter/instances.py:19

The p2p core solves a problem with no analogue in a single-machine program: how to build and keep a trustworthy, self-renewing set of relationships with strangers, on an open network, with no coordinator. It does so by owning every connection in one manager, driving each connection through an explicit `HELLO → PEER-ID → READY` state machine that gates compatibility, then identity, then trust, and dispatching every wire message through the current state's table. Reaching `READY` is not the end of the story — it is the beginning of the conversation that actually matters. Two `READY` peers must now reconcile their copies of the ledger: the joining node may be years behind, and it must download every missing block and transaction in dependency order without ever applying an invalid one. That conversation — `sync-v2` — is Chapter 35.

1. The reactor is Twisted's central event loop: a single-threaded loop that waits for events (network data, timers, connection events) and calls the right callback in response, so the node never blocks on any one connection. Full treatment in Ch. 16. ↩
2. A Twisted Protocol is an object representing one connection's behavior, with callback methods (`connectionMade` , `dataReceived` / `lineReceived` , `connectionLost`) the reactor invokes as the connection's events occur. There is one instance per connection. Full treatment in Ch. 16. ↩
3. A Twisted Factory is an object whose `buildProtocol` method manufactures a fresh Protocol instance for each new connection. Server factories build protocols for inbound connections; client factories for outbound ones. Full treatment in Ch. 16. ↩
4. A Deferred is Twisted's placeholder for a result that is not ready yet — a "future." You attach callbacks (run on success) and errbacks (run on failure); the reactor fires them when the result arrives. Full treatment in Ch. 16. ↩
5. A `LoopingCall` is a Twisted helper that calls a function repeatedly at a fixed interval on the reactor, used here for the periodic connect, reconnect, ping, and sync-rotation timers. Full treatment in Ch. 16 / Ch. 29. ↩
6. A state machine (finite-state machine) is a model in which an object is always in exactly one of a fixed set of states, certain events trigger transitions between states, and each state accepts only a limited set of events. It makes a connection's lifecycle explicit and rejects out-of-phase messages by construction. ↩